UNITRACK — SMART ACADEMIC CALENDAR

Version: 2.0 (B&W; Edition)
Stack: Java 17+, Swing, JDBC, MySQL
Deliverable: Academic Term Submission

Object-Oriented Design & Programming (OODP) Comprehensive Project Plan

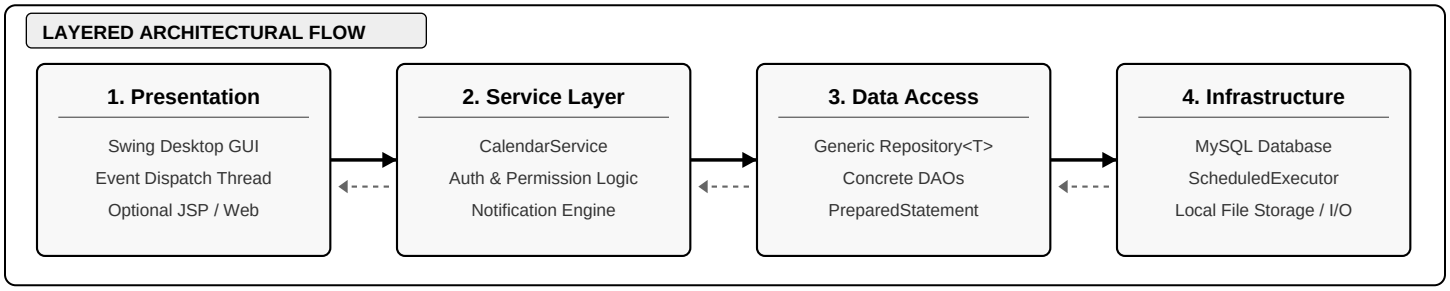
1. PROJECT OVERVIEW & EXECUTIVE SUMMARY

UniTrack — Smart Academic Calendar is a calendar-centric desktop management system engineered for university students, educators, and academic administrators. Rather than treating courses, exams, and notes as disconnected records, UniTrack centers all academic interactions around a dynamic, date-driven calendar interface. The platform bridges high-level semester scheduling with micro-level daily study tasks, providing a unified workspace that complements university learning systems.

Component / Domain	Technology / Framework	OODP / Syllabus Focus	Architectural Role
Language & Runtime	Java SE 17+ (JDK 21 LTS)	Core OOP, Generics, Stream API	Primary application codebase
Presentation Layer	Java Swing (AWT / JFC)	GUI Event Dispatch Thread (EDT)	Interactive month/week/day calendar grid
Persistence & DB	MySQL 8.0+ via JDBC	PreparedStatement, Transactions	Normalized relational academic store
Concurrency Engine	ScheduledExecutorService	Multithreading, Safe State, Locks	Background deadline reminder daemon
Network Integration	Java Sockets / URL Feed API	Networking, I/O Streams, Client-Server	Academic holiday feed / sync module
Web Tier (Optional)	Servlets 5.0 / JSP & Tomcat	MVC Web Pattern, Session State	Auxiliary browser view if mandated

2. SYSTEM ARCHITECTURE & DATA FLOW

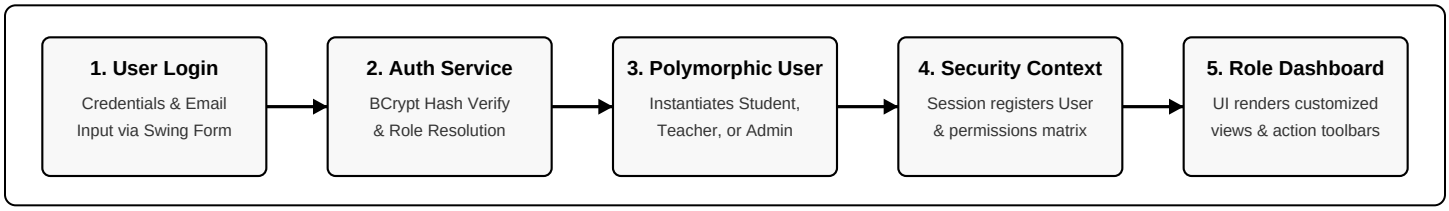
UniTrack employs a strictly decoupled **Multi-Tier Layered Architecture**. This separation of concerns enforces object-oriented maintainability: UI event handlers in Swing never execute raw SQL statements; instead, they communicate through cohesive service boundaries which orchestrate generic data repositories and background threads.



Layer	Key Java Classes / Packages	Primary Responsibilities	Coupling Rules
Presentation	`unitrack.view.*` `unitrack.controller.*`	Renders calendar grid, dialogs, course forms, and file choosers; delegates clicks to controllers.	Calls Service layer only. Must NEVER import java.sql or execute DB queries.
Service (Business)	`unitrack.service.*` `(CalendarService, etc.)`	Implements permission checks, syllabus percentage metrics, event validation, and reminder rules.	Calls Repositories/DAOs; encapsulates all business logic and concurrency triggers.
Data Access (DAO)	`unitrack.repository.*` `(Repository<T, ID>)`	Executes parameterized JDBC queries, manages transaction commit/rollback, maps ResultSet to POJOs.	Isolated behind Generic Repository interfaces; hides database vendor specifics.
Domain Model	`unitrack.model.*` `(User, Event, Course)`	Encapsulates domain entities, enums (Role, EventType, Priority), and validation constraints.	Shared across all layers; pure Java objects with getters/setters and immutability where fit.

3. TARGET USERS & ROLE-BASED ACCESS CONTROL (RBAC)

UniTrack implements strict Role-Based Access Control enforced at the **Service Layer**, ensuring that permission checks do not merely rely on enabling/disabling UI buttons. Below is the authentication and role-routing flowchart:



Feature / Capability	Student	Teacher	Administrator	Security Enforcement Rule
View Academic Calendar	Yes (Enrolled Courses)	Yes (Assigned Courses)	Yes (All Courses / Global)	Filtered by SQL JOIN on user role
Manage Official University Events	No (Read-Only)	No (Read-Only)	Yes (Full CRUD)	Admin-only Service check
Create Lecture / Class Test	No	Yes (Assigned Courses)	Yes (Administrative)	Verify teacher_id == current_user.id
Upload PDF Course Materials	No	Yes (Assigned Courses)	Yes	FileService validates MIME & size
Download Attached Notes	Yes (Enrolled Courses)	Yes (Own Courses)	Yes (Global)	Verify student enrollment in course
Update Syllabus Topic Status	No (View Only)	Yes (Mark complete)	Yes	Teachers only; separate from personal tasks
Manage Personal To-Do Tasks	Yes (Private)	Yes (Private)	Yes (Private)	owner_id enforced on task queries
Configure Semester Dates	No	No	Yes	Service rejects non-admin role

4. CORE FUNCTIONAL MODULES & SPECIFICATIONS

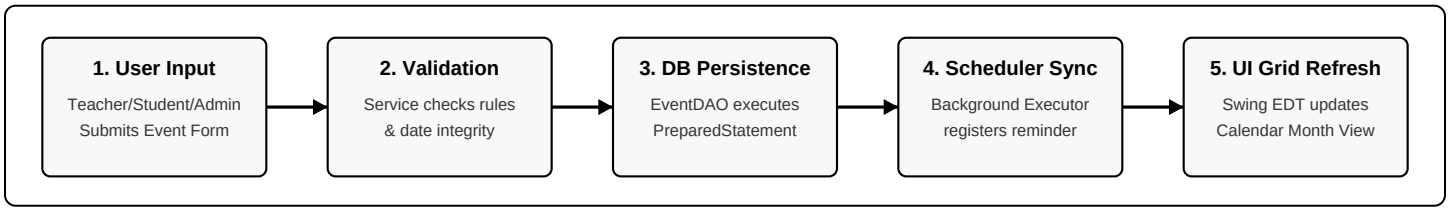
The project is decomposed into 10 cohesive functional modules. All calendar data relies on a unified, normalized `CalendarEvent` model so assessments and lectures do not duplicate scheduling records.

Module	Key Features & Capabilities	UI & Controller Elements	Underlying OODP Mechanism
4.1 Calendar Core	Month grid, Previous/Next/Today navigation, selected date cell highlight, event badges, week/day drilldown.	CalendarPanel, MonthViewGrid, DayDetailDialog, EventBadgeRenderer	Java 8 Date/Time (LocalDate), Custom Swing 2D Graphics Painting
4.2 Academic Events	Lectures, tests, assignments, exams, holidays, personal tasks. Supports time windows, locations, and courses.	EventDialog, EventListPanel, EventFilterComboBox	Polymorphism via EventType enum, Encapsulation of start/end time
4.3 Course Manager	Course codes, titles, assigned teachers, student enrollment mappings, semester bindings.	CourseManageView, EnrollmentDialog, CourseCardComponent	1-to-N and N-to-M entity relationships in OOP and DB
4.4 Notes & Materials	Teachers attach lecture slides/PDFs to specific dates or courses; students download authorized files.	JFileChooser, AttachmentListRenderer, DownloadProgressBar	Java NIO Files, I/O Streams, Storage abstraction via FileService
4.5 Assessments	Quizzes, Midterms, Assignments with instructions and deadlines directly represented on the calendar.	AssessmentDetailView, DeadlineCountBadge, OverdueAlertPanel	Specialized EventType with optional assessment detail records
4.6 Syllabus Progress	Teachers define ordered course topics and toggle completion; calculates exact syllabus progress percentage.	SyllabusTopicList, JProgressBar, TopicCompletionCheckBox	Streams API for counting: completed / total * 100
4.7 Personal To-Do	Private student task management: title, due date, priority (High/Med/Low), completion checkbox.	TodoPanel, TaskCellEditor, TaskFilterTabs (Today/Upcoming)	Collections sorting via Comparator, Separate user-scoped data

Module	Key Features & Capabilities	UI & Controller Elements	Underlying OODP Mechanism
4.8 Semester Analytics	Semester start/end dates, remaining days counter, timeline overview of upcoming milestones.	SemesterBannerPanel, CountdownWidget, SemesterSummaryView	ChronoUnit.DAYS.between(LocalDate.now(), semesterEnd)
4.9 Reminders & Alerts	Automated alerts for approaching tests and assignments (e.g. 24h prior) without duplicate spam.	NotificationTrayIcon, InAppNotificationPopup, AlertBadge	ScheduledExecutorService, Thread-safe deduplication Map
4.10 Search & Filters	Instant search by keyword/title, dynamic filtering by course, date range, or event classification.	SearchTextField with DocumentListener, FilterToolBar	Java Stream.filter() with Predicate lambdas

5. END-TO-END USER WORKFLOWS & PROCESS FLOWCHARTS

Each stakeholder experiences a tailored workflow. Below is the operational lifecycle for scheduling and synchronizing academic events across the platform:



Stakeholder	Step-by-Step Operational Workflow	Key Decisions & Validations
Student Workflow	1. Authenticates or selects demonstration profile. 2. Lands on Monthly Calendar with enrolled course markers. 3. Clicks date cell to open day schedule (lectures, tests, notes). 4. Downloads teacher's uploaded PDF slides for the selected date. 5. Adds personal study task in To-Do list with priority tag. 6. Receives in-app pop-up reminder for tomorrow's assignment. 7. Checks Semester Overview for syllabus completion % vs remaining days.	• Cannot edit teacher's syllabus status. • Personal tasks remain completely private. • Cannot modify official university holiday entries.
Teacher Workflow	1. Logs in and selects assigned course from course switcher. 2. Selects calendar date and clicks 'Add Academic Event'. 3. Configures class test / assignment deadline and attaches PDF worksheet. 4. Updates course syllabus list: checks off 'Inheritance & Polymorphism'. 5. Triggers optional student notification for rescheduled quiz. 6. Reviews course events timeline for upcoming midterm preparation.	• Restricted to courses assigned to teacher_id. • File upload validated for size (< 20MB) and extension. • Syllabus progress auto-recalculates upon save.
Administrator Workflow	1. Logs into Administrative console. 2. Defines semester boundaries (start date, end date, examination weeks). 3. Publishes university-wide holidays and convocation dates. 4. Provisions course offerings, assigns faculty, and enrols students. 5. Audits calendar data integrity across departments.	• Enforces unique course codes & semester dates. • Official events broadcast to ALL calendars. • Cannot be overridden by students or teachers.

6. REQUIRED JAVA & OODP CONCEPT MAPPING

UniTrack is intentionally architected to demonstrate academic mastery of the required Java and Object-Oriented Design curriculum. Concepts are not superficially inserted; each addresses a genuine architectural requirement.

Java / OODP Syllabus Topic	Concrete UniTrack Implementation	Key Java Classes / Interfaces	Software Engineering Justification

Inheritance & Abstraction	Abstract base `User` extended by `Student`, `Teacher`, and `Admin`. Common credentials and role definitions inherited.	`User.java (abstract)`
		`Student.java`
		`Teacher.java`
		`Admin.java`
			Eliminates code duplication; enforces polymorphic role resolution across security boundaries.

Polymorphism & Interfaces	Polymorphic permission checks via `hasPermission(Action)`; domain contracts such as `Notifiable` and `Downloadable`.	`Notifiable.java`
		`Downloadable.java`
		`Searchable.java`
			Enables open-closed principle; reminder engine handles any Notifiable entity seamlessly.

Java / OODP Syllabus Topic	Concrete UniTrack Implementation	Key Java Classes / Interfaces	Software Engineering Justification
Collections Framework	• `List<CalendarEvent>` for chronological event sequences. • `Set<Integer>` for unique student enrollment IDs. • `Map<LocalDate, List<CalendarEvent>>` for instant O(1) day-cell event lookups. • `Queue<Notification>` for alert dispatching.	`ArrayList, HashSet, HashMap, TreeMap, ConcurrentHashMap`	Demonstrates appropriate collection selection based on time complexity and data semantics.
Java 8 Streams & Lambdas	Filtering upcoming tests by course, sorting events by start time, and aggregating completed syllabus topics.	`events.stream() .filter(...) .sorted(...) .collect(...)`	Declarative, readable data transformations without clumsy nested for-loops.
Generics & Generic DAO	Reusable Generic Repository defining standard CRUD operations parameterised by entity type `T` and ID type `ID`.	`Repository<T, ID> AbstractJdbcRepository<T, ID>`	Type safety at compile time; avoids runtime casting and boilerplate DAO code.
Multithreading & Concurrency	Background daemon checking deadline thresholds every 60s; thread-safe event caching; clean JVM shutdown hooks.	`ScheduledExecutorService ReentrantLock AtomicBoolean`	Prevents UI freezing; demonstrates thread safety, deadlock prevention, and graceful shutdown.
Java Networking	Client-server socket module enabling UniTrack client to fetch university-wide calendar feeds or academic holiday data.	`ServerSocket Socket InputStream / OutputStream`	Demonstrates reliable TCP socket communication, timeout handling, and stream parsing.
JDBC & Relational DB	Connecting to MySQL via JDBC driver, executing parameterized queries, transaction control (`commit/rollback`).	`Connection, PreparedStatement ResultSet, Savepoint`	Prevents SQL injection vulnerabilities; ensures atomicity on multi-table updates.
GUI / Java Swing (EDT)	Interactive desktop client with custom painted calendar grids, forms, and dialogs. Background tasks offloaded from EDT.	`SwingWorker<T, V> SwingUtilities.invokeLater JPanel, JTable, JDialog`	Maintains a responsive 60fps UI while performing heavy disk and network I/O.
Servlets / JSP (Optional)	Standalone web module sharing the exact same Service and DAO layers to render calendar events in a web browser.	`CalendarServlet.java calendarView.jsp HttpSession`	Validates enterprise multi-channel architecture sharing common business logic.

7. PROPOSED CLASS DESIGN & DOMAIN MODEL

The class structure strictly follows the Single Responsibility Principle. Below is the taxonomy of core classes across architectural layers:

Layer	Class / Interface	Key Attributes & Fields	Key Methods & Responsibilities
Model (User)	`User (abstract) Student, Teacher, Admin`	`int id; String name; String email; String passwordHash; Role role;`	`hasPermission(Action a); getDashboardView(); getEnrolledCourses();`
Model (Core)	`CalendarEvent`	`int id; String title; EventType type; LocalDateTime start, end; Course course; User creator; String location; String filePath;`	`isUpcoming(); isOverdue(); conflictsWith(CalendarEvent other); hasAttachment();`
Model (Academic)	`Course Semester SyllabusTopic TodoItem`	`code, title, teacherId; startDate, endDate; topicName, isCompleted, order; ownerId, title, priority, isDone;`	`calculateProgress(); markComplete(); isExpired(); getRemainingDays(); sortByPriority();`
Service Layer	`CalendarService CourseService NotificationService FileService`	Repository dependencies, ScheduledExecutor, upload directory path configuration.	`createEvent(User u, Event e); getEventsByDate(LocalDate d); pollApproachingDeadlines(); saveAttachment(File f);`

Layer	Class / Interface	Key Attributes & Fields	Key Methods & Responsibilities
Persistence	`Repository<T, ID>` `EventDao, UserDao, CourseDao, TaskDao`	Database connection credentials, SQL PreparedStatement constants, ResultSet mappers.	`findById(ID id); findAll(); save(T entity); update(T entity); deleteById(ID id); findByDateRange(...);`
Utilities	`DatabaseConnectionPool` `DateUtils, SecurityUtils`	JDBC DataSource, BCrypt hasher, standard DateTimeFormatter patterns.	`getConnection(); hashPassword(pwd); verifyPassword(pwd, hash); formatRange(...);`

8. DATABASE ARCHITECTURE & DATA DICTIONARY

The relational schema is fully normalized (3NF) to eliminate data redundancy while maintaining referential integrity across MySQL tables via foreign keys and cascade rules.

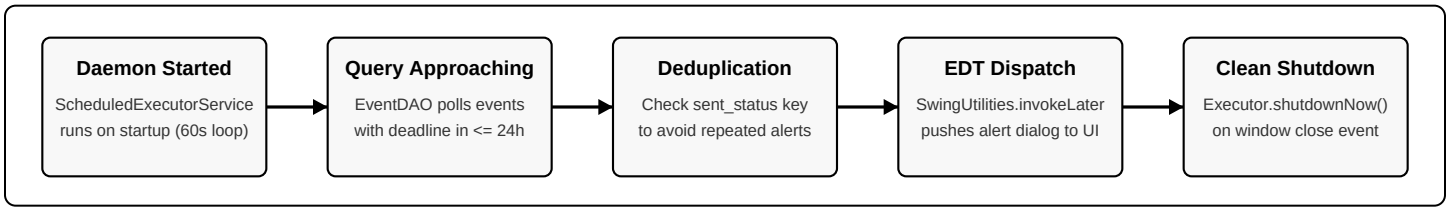
Table Name	Primary Key	Foreign Keys	Key Columns & Types	Description & Constraints
USERS	`id (INT PK AI)`	None	`name VARCHAR(100), email VARCHAR(120), password_hash VARCHAR(255), role VARCHAR(20)`	System users. `email` is UNIQUE. `role` ENUM('STUDENT','TEACHER','ADMIN').
SEMESTERS	`id (INT PK AI)`	None	`name VARCHAR(60), start_date DATE, end_date DATE, is_active BOOLEAN`	Academic terms. Constraint: `CHECK (end_date > start_date)`.
COURSES	`id (INT PK AI)`	`semester_id -> SEMESTERS` `teacher_id -> USERS`	`course_code VARCHAR(20), title VARCHAR(120), credits INT, semester_id INT, teacher_id INT`	Academic courses. `course_code` is UNIQUE within a semester.
ENROLLMENTS	`student_id + course_id`	`student_id -> USERS` `course_id -> COURSES`	`student_id INT, course_id INT, enrolled_at TIMESTAMP DEFAULT NOW()`	Composite PK. Enforces that only users with `STUDENT` role can be enrolled.
CALENDAR_EVENTS	`id (INT PK AI)`	`course_id -> COURSES` `created_by -> USERS`	`title VARCHAR(150), description TEXT, event_type VARCHAR(30), start_dt DATETIME, end_dt DATETIME, location VARCHAR(100)`	Universal calendar entries. Indexed on `(start_dt, end_dt)` for rapid month querying.
NOTES	`id (INT PK AI)`	`course_id -> COURSES` `event_id -> EVENTS` `uploaded_by -> USERS`	`title VARCHAR(120), file_path VARCHAR(255), file_size BIGINT, mime_type VARCHAR(50), uploaded_at TIMESTAMP`	Stores file metadata. Physical file lives in configured local directory; DB stores relative path.
ASSESSMENTS	`id (INT PK AI)`	`event_id (UNIQUE) -> CALENDAR_EVENTS`	`assessment_type VARCHAR(30), instructions TEXT, total_marks INT, weightage DECIMAL(5,2)`	Extended assessment metadata. 1-to-1 link with CALENDAR_EVENTS (prevents event duplication).
SYLLABUS_TOPICS	`id (INT PK AI)`	`course_id -> COURSES` `completed_by -> USERS`	`course_id INT, title VARCHAR(150), topic_order INT, is_completed BOOLEAN, completed_at DATETIME`	Ordered curriculum topics. Teachers check off topics to auto-calculate progress %.
TODO_ITEMS	`id (INT PK AI)`	`owner_id -> USERS`	`owner_id INT, title VARCHAR(150), due_dt DATETIME, priority VARCHAR(10), is_completed BOOLEAN`	Personal student tasks. Strictly private to the owning user. Priority: HIGH, MED, LOW.
NOTIFICATIONS	`id (INT PK AI)`	`recipient_id -> USERS` `event_id -> EVENTS`	`message VARCHAR(255), scheduled_at DATETIME, sent_at DATETIME, status VARCHAR(20), dedup_key VARCHAR(100) UNIQUE`	Alert records. `dedup_key` prevents sending duplicate alerts for the same event window.

9. ADVANCED TECHNICAL SUBSYSTEMS

To fulfill high-scoring academic requirements, UniTrack incorporates two dedicated subsystems: a background multithreaded notification engine and a client-server socket synchronization demonstration.

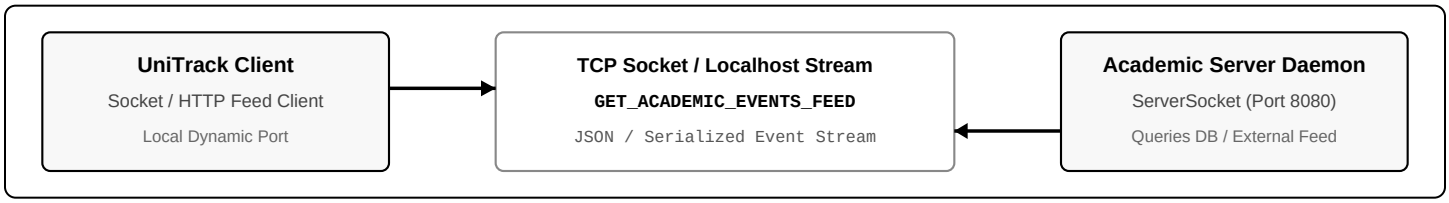
9.1 Multithreaded Background Reminder Engine

The reminder subsystem runs autonomously as a daemon thread. It evaluates upcoming deadlines, verifies deduplication hashes, and uses `SwingUtilities.invokeLater` to safely pass notifications to the Swing Event Dispatch Thread.



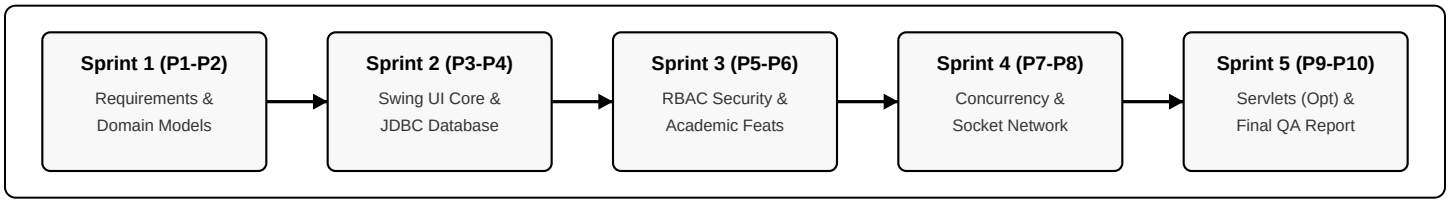
9.2 Client-Server Socket Integration

To meet the Java Networking requirement without relying on external university APIs, UniTrack includes a lightweight academic server daemon. The client socket connects, requests campus-wide holiday feeds, and parses incoming streams into calendar events.



10. 10-PHASE IMPLEMENTATION ROADMAP & SPRINT PIPELINE

Development is sequenced across 10 disciplined phases grouped into 5 iterative sprints, guaranteeing an operational MVP at early stages while methodically expanding into advanced concurrency, networking, and optional web modules.



Phase	Focus & Objective	Key Deliverables & Action Items	OODP Concept	Milestone
Phase 1	Requirements & UI Blueprint	Finalize UI reference from student design; define user stories, use cases, and initial UML class diagrams.	OO Analysis & Design	**Milestone 1 (Spec Complete)**
Phase 2	Project Setup & Domain Models	Establish package hierarchy; build POJO entities (`User`, `CalendarEvent`, `Course`) and enums.	Encapsulation, Enums	**Milestone 2 (Domain Ready)**
Phase 3	Calendar Core (Swing GUI)	Build 7x6 month grid panel; implement previous/next/today navigation; bind date selection to detail pane.	Swing GUI, Event Dispatch	**Milestone 3 (UI Prototype)**
Phase 4	Database Persistence (JDBC)	Design MySQL schema; establish ConnectionPool; write generic `Repository<T, ID>` and EventDAO with PreparedStatement.	Generics, JDBC, SQL	**Milestone 4 (Core MVP)**
Phase 5	RBAC & Course Management	Implement abstract User hierarchy; code Student, Teacher, Admin roles; build course enrollment authorization logic.	Inheritance, Polymorphism	**Milestone 5 (RBAC Live)**

Phase	Focus & Objective	Key Deliverables & Action Items	OODP Concept	Milestone
Phase 6	Academic Features	Integrate PDF attachment uploads/downloads; syllabus topic progress tracking; student private to-do list.	Java NIO, Stream API	**Milestone 6 (Full Academic)**
Phase 7	Concurrency & Notifications	Implement `ScheduledExecutorService` daemon; build deduplicated reminder checking and EDT notification dialogs.	Multithreading, Locks	**Milestone 7 (Alert Engine)**
Phase 8	Java Networking Module	Build client-server TCP socket demonstration for synchronized holiday and university event fetching.	Sockets, I/O Streams	**Milestone 8 (Network Ready)**
Phase 9	Web Module (Servlets/JSP)	If mandated by instructor: create small companion web app sharing the same Service and DAO layers.	Java Servlets, JSP, HTTP	**Milestone 9 (Web Hybrid)**
Phase 10	Testing, QA & Packaging	Execute boundary tests, SQL injection tests, thread safety audits; compile final UML diagrams, slides, and project report.	Software QA & Delivery	**Milestone 10 (Final Submission)**

11. SCOPE MANAGEMENT & MOSCOW PRIORITIZATION

To ensure successful completion before academic deadlines, requirements are categorized using the MoSCoW framework. The team guarantees all **Must-Have (MVP)** deliverables before activating secondary enhancements.

Priority Level	Features & Scope Inclusions	Technical Justification & Risk Mitigation
MUST HAVE (Core MVP)	• Interactive Monthly Calendar Grid with Day Navigation • Create, Read, Update, Delete (CRUD) for Calendar Events • MySQL JDBC persistence with PreparedStatement • Abstract User base with Student & Teacher role separation • Course association and Event Type filtering • Personal To-Do task checklist with persistence	Essential core without which the project cannot be demonstrated. Fulfills the minimum OODP passing criteria.
SHOULD HAVE (High Priority)	• Notes and PDF attachment upload/download system • Teacher-managed syllabus topic list with % calculation • Multithreaded background reminder engine (ScheduledExecutor) • TCP Socket / URL academic holiday sync demonstration • Stream API sorting, filtering, and metric calculation	Fulfills advanced course requirements (Multithreading, Networking, Collections, Streams) for maximum grading marks.
COULD HAVE (Nice to Have)	• Week View and Day View calendar representations • Administrator console for global holiday scheduling • Companion Servlet/JSP browser view (if requested by instructor) • Dark/Light visual theme switcher in Swing	Added only after the core desktop workflow and database operations are 100% stable and verified.
WON'T HAVE (Out of Scope)	• Bi-directional live sync with Google Calendar API / Outlook • Real-time automated assignment grading / code evaluation • Online payment or university fee processing gateways	Prevents scope creep; keeps project focused on OODP principles rather than external third-party cloud infrastructure.

12. QUALITY ASSURANCE & TESTING CHECKLIST

A rigorous testing plan validates edge cases, boundary conditions, thread safety, and security vulnerabilities across all functional units:

Subsystem	Test Scenario & Test Case	Input / Condition	Expected System Outcome
Calendar Navigation	Year boundary rollover and leap year handling	Navigate December 2026 -> January 2027; Feb 2028	Correct month name, 31 days in Jan, 29 days in leap Feb, correct weekday alignment.
RBAC Security	Student attempts unauthorized syllabus topic modification	Student role sends updateTopicCompletion() request	Service throws SecurityException; UI displays access-denied error dialog.
Data Integrity	SQL Injection prevention on search and input fields	Enter '" OR '1'='1" in search box or event title	PreparedStatement treats input as literal text; no syntax error or unauthorized data leak.

Subsystem	Test Scenario & Test Case	Input / Condition	Expected System Outcome
File Handling	Missing file or corrupted attachment download	Delete physical file from disk; click Download in UI	FileService detects missing file gracefully; logs warning; alerts user without crashing.
Concurrency	Background reminder daemon during application exit	User closes main window while executor thread is active	WindowListener calls executor.shutdownNow(); background thread terminates immediately.
Networking	Calendar synchronization server unavailable	Attempt sync when ServerSocket daemon is offline	SocketTimeoutException caught; informs user offline mode active; calendar stays usable.

13. FINAL DELIVERABLES & SUBMISSION PACKAGING

Final academic submission package is organized strictly according to standard Java project conventions:

Deliverable Item	Format / Artifact	Contents & Description
Complete Source Code	Clean Java Project (.zip)	Fully commented, production-grade Java source code organized into layered packages.
Database DDL & Demo Data	schema.sql + sample_data.sql	DDL script creating all 10 tables, indexes, constraints, and populated demo datasets.
UML Architecture Diagrams	PDF / High-Res PNG	Complete Use-Case Diagram, Class Hierarchy Diagram, and Relational Entity-Relationship Diagram.
Setup & Execution Guide	README.md	Prerequisites (JDK, MySQL), DB setup commands, compile instructions, and login credentials.
OODP Concept Mapping Document	Academic Report Section	Detailed written justification explaining how each required OOP concept was practically utilized.
Presentation Slides & Script	PowerPoint / PDF + Demo Script	10-slide deck summarizing problem, architecture, OODP demonstration, and live demo script.

Recommended Package Structure:

```
unitrack/
  [+] model/         Domain POJOs: User, Student, Teacher, Admin, CalendarEvent, Course, Semester, SyllabusTopic, TodoItem,
Notification
  [+] view/          Swing GUI: MainCalendarFrame, MonthGridPanel, DayDetailDialog, EventFormDialog, CourseView, TodoPanel
  [+] controller/    Event Controllers: CalendarController, AuthController, CourseController, TaskController
  [+] service/       Business Logic: CalendarService, AuthService, CourseService, NotificationService, FileService,
ProgressService
  [+] repository/    Data Persistence: Repository<T, ID>, EventDao, UserDao, CourseDao, TaskDao, NoteDao
  [+] database/      Infrastructure: ConnectionPool, DatabaseConfig, MigrationRunner
  [+] concurrency/   Threads: ReminderDaemon, ThreadFactoryHelper
  [+] networking/    Sockets: CalendarSyncClient, AcademicServerDaemon
  [+] util/          Helpers: DateUtils, SecurityUtils, InputValidator, Constants
```